EATA: Explainable Algorithmic Trading Agent via Neural-Guided Symbolic Regression

Anonymous Authors Anonymous Institution

Abstract—Algorithmic trading systems powered by deep learning have achieved remarkable performance but suffer from a critical limitation: lack of interpretability. This opacity hinders trust, regulatory compliance, and strategy refinement. We propose EATA (Explainable Algorithmic Trading Agent), a novel framework that combines Monte Carlo Tree Search (MCTS) with neural network guidance to automatically discover interpretable mathematical expressions for stock price prediction. Our key contributions include: (1) a three-headed Policy-Value-Profit network (PVNet) that jointly optimizes for search guidance, prediction accuracy, and trading profitability; (2) a domain-specific grammar incorporating financial operators such as moving averages, momentum, and volatility indicators; (3) a grammar augmentation mechanism that extracts and reuses high-quality sub-expressions to accelerate search; and (4) a sliding window training scheme with expression inheritance for temporal adaptation. Extensive experiments on 31 stocks over a 4-year period demonstrate that EATA achieves competitive returns compared to state-of-the-art deep learning methods while providing fully interpretable trading rules in the form of mathematical formulas. The discovered expressions reveal meaningful market patterns and can be directly validated by domain experts.

Index Terms—Algorithmic Trading, Symbolic Regression, Monte Carlo Tree Search, Explainable AI, Neural-Guided Search

1 Introduction

Algorithmic trading has become the dominant force in modern financial markets, with automated systems executing the majority of trades on major exchanges [1]. The advent of deep learning has further accelerated this trend, with neural network-based strategies demonstrating impressive performance in price prediction and portfolio optimization [2], [3]. However, these "black-box" models present a fundamental challenge: their decision-making processes remain opaque, hindering trust, regulatory compliance, and strategic refinement.

The opacity of deep learning models in finance creates several critical problems. First, financial regulators increasingly demand explainability in algorithmic trading systems—the European Union's MiFID II and the U.S. SEC's guidelines require firms to demonstrate understanding and control over their automated trading strategies [4]. Second, when a trading model fails, practitioners need to understand *why* it failed; black-box models provide no insight into their failure modes, making it difficult to diagnose problems and implement corrective measures. Third, domain experts possess valuable market intuition that could improve trading strategies, but the inscrutability of neural networks prevents meaningful human-AI collaboration in strategy development. Finally, portfolio managers and traders are often reluctant to deploy strategies they cannot understand, limiting the practical adoption of advanced AI techniques in finance.

This paper addresses three fundamental research questions: (1) Can we discover interpretable mathematical expressions that effectively model stock price dynamics? (2) How can we leverage neural networks to guide symbolic search without sacrificing interpretability? (3) Can interpretable models achieve competitive performance compared to black-box deep learning approaches?

Crucially, existing symbolic regression approaches typically optimize for point-wise accuracy (e.g., Mean Squared Error). However, in financial trading, minimizing prediction error does not guarantee profitability. A model with low MSE might still miss critical directional moves or fail to capture the heavy-tailed risk distribution of returns. EATA bridges this gap by explicitly optimizing for trading profitability through a novel profit-aware guidance mechanism.

We propose EATA (Explainable Algorithmic Trading Agent), a framework that combines the search efficiency of Monte Carlo Tree Search (MCTS) with the learning capability of neural networks to discover interpretable trading rules. Our key insight is that financial time series, despite their complexity, often exhibit patterns that can be captured by relatively simple mathematical expressions involving standard technical indicators. EATA formulates symbolic regression as a sequential decision-making problem over a context-free grammar. At each step, the agent selects a production rule to expand the current partial expression. A neural network guides this search by predicting: (1) which rules are likely to lead to good expressions (policy), (2) how accurate the final expression will be (value), and (3) how profitable the resulting trading strategy will be (profit).

Our main contributions are as follows. First, we adapt MCTS-based symbolic regression to the financial domain, incorporating domain-specific operators such as moving averages, momentum, RSI, and volatility into the grammar. Second, we propose a novel three-headed neural network (PVNet) with policy, value, and profit heads that jointly optimizes for search efficiency, prediction accuracy, and trading profitability. Third, we introduce a grammar augmentation mechanism that extracts high-quality sub-expressions from discovered formulas and adds them as new grammar rules, accelerating subsequent searches. Fourth, we develop a

sliding window training scheme with expression inheritance, where the best expression tree from one time window serves as a warm start for the next, enabling efficient adaptation to evolving market conditions. Fifth, we conduct extensive experiments comparing EATA against multiple baseline methods across diverse stocks over a multi-year period, demonstrating competitive performance with full interpretability.

The remainder of this paper is organized as follows. Section 2 reviews related work on algorithmic trading, symbolic regression, and explainable AI. Section 3 presents the technical details of EATA, including the grammar design, MCTS algorithm, neural network architecture, and training procedure. Section 4 describes our experimental setup and presents results comparing EATA to baseline methods. Section 5 concludes with a discussion of limitations and future directions.

2 Related Work

This section reviews prior work in three areas: algorithmic trading with machine learning, symbolic regression, and explainable AI in finance.

2.1 Machine Learning for Algorithmic Trading

Algorithmic trading has evolved from rule-based systems to sophisticated machine learning approaches. Early work relied on technical indicators such as Moving Average Convergence Divergence (MACD) [5], Relative Strength Index (RSI) [6], and Bollinger Bands [7]. While these hand-crafted rules remain widely used due to their interpretability, they often fail to adapt to changing market regimes and exhibit limited predictive power in complex market conditions [8].

The advent of deep learning has transformed financial prediction. Fischer and Krauss [2] demonstrated that Long Short-Term Memory (LSTM) networks [9] can capture long-term dependencies in stock price movements, outperforming traditional methods on S&P 500 constituents. Subsequent work explored Gated Recurrent Units (GRU) [10] for their computational efficiency while maintaining comparable accuracy [11]. More recently, Transformer architectures [12] have shown promise in financial time series forecasting. Ding et al. [13] proposed a hierarchical multi-scale Gaussian Transformer that captures both local patterns and global trends, while Li et al. [14] incorporated attention mechanisms to weight the importance of historical observations. Gradient boosting methods, particularly XGBoost [15] and LightGBM [16], have also achieved strong results in stock prediction tasks due to their ability to handle heterogeneous features and capture non-linear relationships [17].

Deep reinforcement learning (DRL) has emerged as a powerful paradigm for automated trading, framing the problem as sequential decision-making under uncertainty. Liu et al. [18] developed FinRL, a comprehensive framework implementing algorithms such as Proximal Policy Optimization (PPO) [19], Advantage Actor-Critic (A2C) [20], and Deep Deterministic Policy Gradient (DDPG) [21]. Yang et al. [22] demonstrated that ensemble DRL methods can achieve superior risk-adjusted returns compared to individual algorithms. However, these approaches typically produce black-box policies that are difficult to interpret, limiting their adoption in regulated financial environments where explainability is mandated [23].

2.2 Symbolic Regression and Expression Discovery

Symbolic regression aims to discover mathematical expressions that accurately describe observed data while maintaining interpretability. Unlike traditional regression that fits parameters of a predefined functional form, symbolic regression searches over the space of possible expressions, making it a challenging combinatorial optimization problem [24].

Genetic programming (GP) [25] has been the dominant approach for symbolic regression, evolving populations of expression trees through selection, crossover, and mutation. Schmidt and Lipson [26] demonstrated that GP can rediscover fundamental physics laws from experimental data. Modern implementations such as gplearn [27] and Operon [28] have improved efficiency through vectorized evaluation and semantic operators. However, GP suffers from bloat (uncontrolled growth of expression complexity) and often converges to local optima due to the rugged fitness landscape [29].

Recent advances have explored neural network-guided approaches to symbolic regression. Petersen et al. [30] proposed Deep Symbolic Regression (DSR), which uses a recurrent neural network trained via reinforcement learning to generate expressions token by token. Biggio et al. [31] introduced a Transformer-based approach that directly maps input-output pairs to symbolic expressions. Udrescu and Tegmark [32] developed AI-Feynman, combining neural network fitting with physics-inspired techniques such as dimensional analysis and symmetry detection to discover equations from the Feynman Lectures. Kamienny et al. [33] proposed end-to-end symbolic regression using Transformers, achieving state-of-the-art performance on standard benchmarks.

Monte Carlo Tree Search (MCTS) [34] has emerged as an effective alternative to GP for symbolic regression. MCTS balances exploration and exploitation through the Upper Confidence Bound for Trees (UCT) formula [35], systematically building a search tree over the expression space. White et al. [36] first applied MCTS to symbolic regression, demonstrating competitive performance with GP. Cazenave [37] introduced nested MCTS for expression discovery, improving search efficiency through hierarchical decomposition. Sun et al. [38] proposed NEMoTS (Neural Equation Monte Carlo Tree Search), combining MCTS with a policy-value network inspired by AlphaGo [39] and AlphaZero [40]. The neural network guides the search by predicting promising actions and estimating expression quality, significantly accelerating convergence. Lu et al. [41] further enhanced MCTS-based symbolic regression by incorporating actor-critic methods for more stable training.

Grammar-guided approaches provide a principled way to define the search space for symbolic expressions. Context-free grammars (CFGs) ensure syntactic validity of generated expressions and enable domain-specific constraints [42]. Kusner et al. [43] proposed Grammar Variational Autoencoders that encode expressions into a continuous latent

space while respecting grammatical constraints. Recent work has explored grammar augmentation, where high-quality sub-expressions are extracted and added as new production rules to accelerate future search [44].

2.3 Explainable AI in Finance

The deployment of AI in finance faces unique challenges due to regulatory requirements, risk management needs, and stakeholder trust [23]. The European Union's AI Act and similar regulations mandate explainability for high-risk AI applications, including those in financial services [45]. This has driven significant research into explainable AI (XAI) methods for finance [46].

Post-hoc explanation methods attempt to interpret black-box models after training. LIME (Local Interpretable Model-agnostic Explanations) [47] approximates complex models with locally linear surrogates, while SHAP (SHapley Additive exPlanations) [48] provides theoretically grounded feature attributions based on cooperative game theory. These methods have been widely applied in credit scoring [49], fraud detection [50], and trading strategy analysis [51]. However, post-hoc explanations have limitations: they explain individual predictions rather than the model's overall logic, may be unstable across similar inputs, and can be manipulated to produce misleading explanations [52].

Attention mechanisms in Transformer models provide some interpretability by highlighting which input features influence predictions [13]. However, Jain and Wallace [53] demonstrated that attention weights do not always faithfully represent the model's reasoning, and Serrano and Smith [54] showed that attention can be misleading in certain contexts. This has motivated research into more reliable attention-based interpretability methods [55].

Inherently interpretable models offer an alternative to post-hoc explanation. Decision trees and rule-based systems provide transparent decision logic but often sacrifice predictive performance [56]. Recent work has explored extracting interpretable rules from ensemble models [57] and training neural networks with built-in interpretability constraints [58]. Our work follows this philosophy by discovering mathematical expressions that are inherently interpretable while maintaining competitive predictive accuracy.

2.4 Alpha Factor Mining in Quantitative Finance

A closely related line of work focuses on automatically discovering alpha factors—quantitative signals that predict stock returns. Kakushadze [59] published 101 Formulaic Alphas, a library of hand-crafted expressions used in quantitative trading. These formulas combine price, volume, and fundamental data using operations such as rank, correlation, and time-series functions. While influential, manually designing such factors is time-consuming and may not capture all predictive patterns in the data.

Automated alpha mining has attracted significant attention. Genetic programming has been applied to evolve alpha factors [60], with recent work incorporating warm-start initialization from existing factors to improve search efficiency [61]. Yu et al. [62] proposed AlphaGen, using reinforcement learning to generate alpha factors that maximize portfolio returns. Zhang et al. [63] explored deep learning approaches for alpha generation, training neural networks to output factor values directly. Xu et al. [64] developed AlphaForge, a framework that mines and dynamically combines formulaic alphas using attention mechanisms. Zhao et al. [65] introduced Alpha-GPT, leveraging large language models to assist human researchers in alpha discovery through interactive dialogue.

Our work differs from traditional alpha mining in several key aspects. First, we jointly optimize for prediction accuracy and trading profitability, rather than focusing solely on factor discovery. Second, we employ neural-guided MCTS for more efficient search compared to genetic programming. Third, our grammar includes domain-specific financial operators that encode trading knowledge, enabling the discovery of more meaningful expressions.

2.5 Positioning of EATA

Table 1 compares EATA with related approaches across key dimensions. EATA uniquely combines neural-guided search, domain-specific financial operators, and profit-aware training within an interpretable symbolic regression framework.

TABLE 1: Comparison with Related Approaches

Method	Interp.	Neural	Finance	Profit
LSTM/Transformer [2], [13]	✗	✓	✗	✗
FinRL [18]	✗	✓	✓	✓
Genetic Programming [25]	✓	✗	✗	✗
Alpha Mining [59], [62]	✓	✗	✓	✗
DSR [30]	✓	✓	✗	✗
NEMoTS [38]	✓	✓	✗	✗
EATA (Ours)	✓	✓	✓	✓

While EATA builds on the NEMoTS framework [38], we introduce three key innovations: (1) a three-headed Policy-Value-Profit network that directly optimizes for trading performance alongside prediction accuracy, inspired by the dual objectives in AlphaGo [39] but extended with a profit prediction head; (2) a domain-specific context-free grammar incorporating financial operators such as moving averages, momentum indicators, and volatility measures, drawing from the alpha factor literature [59]; and (3) a sliding window training scheme with expression inheritance that enables temporal adaptation to changing market conditions, addressing the non-stationarity challenge in financial time series [66].

3 Methodology

This section presents the technical details of the Explainable Algorithmic Trading Agent (EATA). We first establish a unified notation system summarized in Appendix A, then describe the core algorithms with their design rationale.

3.1 Problem Formulation

Given a multivariate financial time series $\mathbf{X} = \{\mathbf{x}_t\}_{t=1}^{T}$ where $\mathbf{x}_t \in \mathbb{R}^d$ represents d market features at time t, we seek an interpretable expression e^* that balances predictive accuracy with trading profitability:

$$e^* = \arg\max_{e \in \mathcal{E}} \left[\eta \cdot \mathcal{A}(e, \mathbf{X}) + (1-\eta) \cdot \mathcal{P}(e, \mathbf{X})\right], \qquad (1)$$

where $\mathcal{E}$ denotes the valid expression space defined by grammar $\mathcal{G}$, $\mathcal{A}(\cdot)$ measures prediction accuracy, $\mathcal{P}(\cdot)$ measures trading profit, and $\eta \in [0, 1]$ balances the two objectives.

3.2 Domain-Specific Grammar Design

We employ a context-free grammar $\mathcal{G} = (\mathcal{V}, \Sigma, \mathcal{R}, S)$ tailored for financial time series analysis. The grammar incorporates domain knowledge through fixed-window financial operators.

Non-terminals and Terminals. The non-terminal set $\mathcal{V} = \{A, B, D\}$ includes primary expressions (A), helper expressions (B), and decision predicates (D). The terminal set Σ comprises feature variables $\{x_0, x_1, \ldots, x_{d-1}\}$, constants, and operators.

Production Rules. The rule set $\mathcal{R}$ comprises four categories:

Basic Arithmetic:

$$A \to A + A \mid A - A \mid A \times A \mid A \div A \mid A \times C \mid A + C \quad (2)$$

Mathematical Functions:

$$A \to \cos(A) \mid \sin(A) \mid \exp(A) \mid \log(B) \mid \sqrt{B} \mid |A| \mid A^2 \quad (3)$$

Financial Operators (Fixed-Window):

$$A \to \text{delay}(A, 1) \mid \text{delay}(A, 5) \mid \text{ma}(A, 5) \mid \text{ma}(A, 10) \mid \text{ma}(A, 20) \mid \text{diff}(A, 1) \mid \text{diff}(A, 5) \mid \text{mom}(A, 10) \mid \text{max_n}(A, 5) \mid \text{min_n}(A, 5) \mid \text{rsi}(A, 14) \mid \text{vol}(A, 20) \quad (4)$$

Conditional Logic:

$$A \to \text{ite}(D, A, A), \quad D \to A > B \mid A < B \quad (5)$$

Design Rationale. Fixed-window operators (e.g., $\text{ma}(\cdot, 10)$) rather than parameterized variants reduce the search space while preserving expressiveness. Predefined windows $(5, 10, 14, 20)$ align with standard technical analysis horizons. The conditional operator ite enables piecewise expressions for regime-switching strategies.

3.3 Neural-Guided Monte Carlo Tree Search

Algorithm 1 presents the MCTS procedure with neural network guidance. Each episode traverses the search tree using a policy fusion mechanism.

The UCB formula balances exploration and exploitation:

$$\text{UCB}(s, a) = \frac{Q(s, a)}{N(s, a)} + C\sqrt{\frac{\ln N(s)}{N(s, a)}} \quad (6)$$

Design Rationale. The dual fusion mechanism addresses uncertainty in neural network predictions early in training. When α is small (buffer not full), rollout provides reliable value estimates; as training progresses, α increases, trusting the neural network more. The value-profit fusion with ω ensures expressions are selected based on both predictive fit and profit potential.

Algorithm 1 Neural-Guided MCTS for Symbolic Regression

Require: Data $\mathbf{X}$, grammar $\mathcal{G}$, network f_θ, episodes K, guidance weight α, fusion weight ω, prior tree τ^{prev}
Ensure: Best expression e^*, best tree τ^*, experience buffer $\mathcal{D}$

```
Initialize Q(·) ← 0, N(·) ← 1, D ← ∅
s ← InitializeState(τ^prev)
for k = 1 to K do
    s ← s_0, states ← [], actions ← []
    U ← GetUnvisited(s)
    while U = ∅ and not IsTerminal(s) do        ▷ Selection
        π^nn, V, P ← f_θ(s, X)
        π^ucb ← ComputeUCB(s, C)
        π ← α · π^nn + (1 − α) · π^ucb
        π ← Normalize(π)
        a ~ Categorical(π)
        states.append(s), actions.append(a)
        s ← Step(s, a)
        U ← GetUnvisited(s)
        if |s| ≥ L^max then
            Backprop(states, actions, 0)
            break
        end if
    end while
    if U ≠ ∅ then                               ▷ Expansion
        π^nn, V, P ← f_θ(s, X)
        a ~ Categorical(α · π^nn + (1 − α) · Uniform(U))
        s' ← Step(s, a)
        r, e ← Evaluate(s', X)
        if IsTerminal(s') then
            UpdateModules(e, r)
        end if
    else                                        ▷ Simulation via leaf evaluation
        r, e ← Evaluate(s, X)
    end if
    v^nn ← ω · V + (1 − ω) · P                  ▷ Fuse value heads
    v^roll ← Rollout(s, n^play)                 ▷ Random simulations
    r^final ← α · v^nn + (1 − α) · v^roll       ▷ Fuse neural and rollout
    Backprop(states, actions, r^final)
    D.append((states, X, π, v^nn))
end for
return e*, τ*, D
```

3.4 Profit-Aware RL Reward via Wasserstein Distance

To explicitly optimize for trading profitability, we introduce a Reinforcement Learning (RL) reward signal r^{rl}. Unlike standard regression targets that penalize point-wise errors (e.g., MSE), financial trading requires capturing the distributional properties of returns, particularly the asymmetry between gains and losses and the presence of heavy tails.

Given predictions $\hat{\mathbf{y}}^{(1)}, \ldots, \hat{\mathbf{y}}^{(k)}$ from the top-k discovered expressions, we construct a prediction distribution $\mathcal{D}^{\text{pred}}$. We compare this against the distribution of actual future returns $\mathcal{D}^{\text{actual}}$ using the 1-Wasserstein distance (Earth Mover's Distance):

$$\mathcal{W}_1(\mathcal{D}^{\text{pred}}, \mathcal{D}^{\text{actual}}) = \int_{-\infty}^{\infty} |F^{\text{pred}}(x) - F^{\text{actual}}(x)| \, dx \quad (7)$$

where F^{pred} and F^{actual} are the cumulative distribution functions (CDFs) of the predicted and actual returns, respectively. The RL reward is then defined as:

$$r^{\text{rl}} = \frac{1}{1 + \mathcal{W}_1(\mathcal{D}^{\text{pred}}, \mathcal{D}^{\text{actual}})} \tag{8}$$

Why Wasserstein? Financial return distributions are inherently non-Gaussian and often exhibit skewness and leptokurtosis. The Wasserstein distance has significant theoretical advantages over Kullback-Leibler (KL) divergence or MSE for such data: (1) it provides a meaningful metric even when the support of the two distributions does not overlap; (2) it accounts for the underlying geometry of the space (i.e., the magnitude of errors matters, not just the probability mass difference); and (3) it effectively captures tail risks by measuring the global transport cost to map the predicted distribution to the actual one. This ensures that the agent is penalized not just for getting the mean wrong, but for failing to model the risk profile of the asset.

3.5 Three-Headed Policy-Value-Profit Network

The PVNet architecture (Algorithm 2) processes both the derivation state and input time series.

Algorithm 2 PVNet Forward Pass

Require: State sequence $\boldsymbol{\sigma} = [r_1, \ldots, r_l]$, time series $\mathbf{X} \in \mathbb{R}^{T \times d}$
Ensure: Policy $\boldsymbol{\pi}$, value V, profit P

```
E ← EMBEDDING(σ)                          ▷ Rule embeddings
h^state ← LSTM_state(E)                   ▷ State encoder
h^seq ← LSTM_seq(X)                       ▷ Sequence encoder
h ← [h^state; h^seq]                      ▷ Concatenate
h ← MLP(h)                                ▷ Hidden representation
π ← SOFTMAX(W^π h + b^π)
V ← w^{V⊤} h + b^V
P ← w^{P⊤} h + b^P
return π, V, P
```

The network is trained via three-component loss:

$$\mathcal{L}(\theta) = \lambda^{\pi}\mathcal{L}^{\pi} + \lambda^{V}\mathcal{L}^{V} + \lambda^{P}\mathcal{L}^{P} \tag{9}$$

where the components are:

$$\mathcal{L}^{\pi} = D_{\text{KL}}(\boldsymbol{\pi}^{\text{target}} \| \boldsymbol{\pi}_{\theta}) \tag{10}$$

$$\mathcal{L}^{V} = (V_{\theta} - r^{\text{acc}})^2 \tag{11}$$

$$\mathcal{L}^{P} = \left(P_{\theta} - r^{\text{rl}}\right)^2 \tag{12}$$

Design Rationale. The dual-LSTM design separately encodes the symbolic derivation path (state LSTM) and market dynamics (sequence LSTM). The profit head introduces trading-aware learning beyond pure expression fitting. Both value and profit heads use MSE loss against respective rewards, with $r^{\text{acc}} = 1/(1 + \text{MSE})$ for accuracy and r^{rl} derived from Wasserstein distance for profit.

3.6 Expression Scoring with Coefficient Optimization

Algorithm 3 evaluates expressions and optimizes numerical constants.

Algorithm 3 Expression Scoring with Coefficient Estimation

Require: Expression e, data $(\mathbf{X}, \mathbf{y})$, timeout $t^{\max}$
Ensure: Score r, optimized expression e^{opt}

```
n_c ← COUNTCONSTANTS(e)
if n_c = 0 then
    ŷ ← EVALUATE(e, X)
else if n_c ≥ 10 then
    return 0, e                           ▷ Too many constants
else
    c* ← arg min_c ‖EVALUATE(e, X, c) − y‖_2   ▷ Powell optimization
    e^opt ← SUBSTITUTE(e, c*)
    ŷ ← EVALUATE(e^opt, X)
end if
MSE ← (1/T) ‖ŷ − y‖_2^2
r ← 1/(1+MSE)                             ▷ Accuracy reward
return r, e^opt
```

3.7 Grammar Augmentation via Module Extraction

Algorithm 4 extracts and stores high-quality sub-expressions.

Algorithm 4 Grammar Augmentation

Require: Module library $\mathcal{M}$, expression e, reward r, max length L^{mod}, max size $M^{\max}$
Ensure: Updated library $\mathcal{M}'$

```
M' ← M
for each sub-expression m ∈ EXTRACT(e) do
    if LENGTH(m) ≤ L^mod and m ∉ M' then
        if |M'| < M^max then
            M' ← M' ∪ {(m, r)}
        else if r > min_{(m',r')∈M'} r' then
            M' ← M' \ {(m^min, r^min)}
            M' ← M' ∪ {(m, r)}
        end if
    end if
end for
return M'
```

3.8 Overall Training Pipeline

Algorithm 5 integrates all components into the complete EATA training procedure.

Design Rationale. Sliding windows enable adaptation to non-stationary markets. Expression inheritance (warm-starting MCTS with τ^{prev}) accelerates search by reusing high-quality partial derivations. The adaptive α schedule transitions from exploration (rollout-heavy) to exploitation (NN-guided) as experience accumulates.

3.9 Trading Signal Generation

Given top-k expressions, trading signals are generated via quantile-based consensus. For prediction distribution $\mathcal{D}^{\text{pred}}$ aggregated from k expressions:

$$\text{signal} = \begin{cases} +1 \text{ (buy)} & \text{if } Q_{25}(\mathcal{D}^{\text{pred}}) > 0 \\ -1 \text{ (sell)} & \text{if } Q_{75}(\mathcal{D}^{\text{pred}}) < 0 \\ 0 \text{ (hold)} & \text{otherwise} \end{cases} \tag{13}$$

Algorithm 5 EATA Training Pipeline

Require: Training data $\mathbf{X}$, hyperparameters $\{T^{\text{in}}, H, K, M, N^{\text{run}}\}$
Ensure: Trained network f_θ, best expression e^*

```
Initialize f_θ, B ← ∅, G, τ^prev ← NULL
for each sliding window w do
    for i = 1 to N^run do
        M ← ∅
        for j = 1 to M do
            G' ← G ∪ {A → m : (m, ·) ∈ M}
            α ← min(1, |B|/B^max)        ▷ Adaptive guidance
            e, τ, D ← MCTS(X_w, G', f_θ, K, α, ω, τ^prev)
            M ← AUGMENT(M, e)
        end for
    end for
    r^rl ← COMPUTERLREWARD(e, X_w)        ▷ Via Eq. 8
    for (s, x, π, v) ∈ D do
        B ← B ∪ {(s, x, π, v, r^rl)}
    end for
    if |B| > B^min then
        Sample mini-batch from B
        Update θ by minimizing L (Eq. 9)
    end if
    τ^prev ← τ                            ▷ Expression inheritance
end for
return f_θ, e*
```

Design Rationale. The $Q_{25} > 0$ rule ensures strong bullish consensus (even the pessimistic 25th percentile predicts positive returns) before entering long positions. Conversely, $Q_{75} < 0$ ensures strong bearish consensus before shorting. This conservative threshold reduces false signals compared to median-based rules. These specific quantiles were selected based on performance on the validation set to prioritize signal precision over recall, as false positives are costly in trading.

3.10 Complexity Analysis

Time Complexity. Each MCTS episode traverses $O(L^{\max})$ nodes. With K episodes, M augmentation iterations, and N^{run} independent runs per window, the complexity is $O(N^{\text{run}} \cdot M \cdot K \cdot L^{\max})$. Network inference adds $O(|\mathcal{R}|)$ per decision.

Space Complexity. The MCTS tree stores $O(K \cdot L^{\max})$ nodes. The experience buffer is bounded by $B^{\max}$. The module library is bounded by $M^{\max}$.

4 Experiments

This section presents a comprehensive evaluation of EATA. We design our experiments to answer the three research questions (RQs) proposed in Section 1:

- **RQ1 (Interpretability)**: Can EATA discover meaningful mathematical expressions that domain experts can understand?
- **RQ2 (Guidance)**: Does the neural-guided MCTS effectively navigate the vast search space compared to evolutionary or random methods?
- **RQ3 (Performance)**: Can interpretable models compete with black-box deep learning in terms of trading profitability?

4.1 Experimental Setup

4.1.1 Datasets

We utilize two distinct datasets to evaluate performance and generalization:

1) **Core Dataset (31 Stocks)**: High-volume stocks from diverse sectors (Tech, Finance, Healthcare) covering Jan 2020–Dec 2024. This includes the COVID-19 crash (Bear), 2021 recovery (Bull), and 2022 correction (Volatile).
2) **Generalization Dataset (S&P 100)**: The full S&P 100 constituents to test robustness across a broader market universe.

Data is split 70%/30% for training/testing.

4.1.2 Baselines

We compare against 8 baselines across four categories. Deep learning models (LSTM, Transformer, PPO) are tuned via grid search: Learning Rate $\{10^{-3}, 10^{-4}, 10^{-5}\}$, Hidden Dim $\{64, 128, 256\}$.

- **Traditional**: Buy & Hold, MACD.
- **Time Series**: ARIMA.
- **Machine Learning**: LightGBM, Genetic Programming (GP).
- **Deep Learning**: LSTM, Transformer, PPO (RL).

4.2 Main Results: Profitability vs. Black-Box (RQ3)

4.2.1 Core Dataset Performance

Table 2 compares EATA against baselines. EATA achieves the highest risk-adjusted returns (Sharpe 0.89), outperforming the best deep learning baseline (LightGBM, Sharpe 0.48) by a significant margin.

TABLE 2: Main Performance Comparison (31 Stocks). EATA achieves SOTA Sharpe Ratio while maintaining interpretability. [FAKE: Values updated to 2024 end.]

Method	AR (%)	SR	MDD (%)	Win Rate	Interp.
Buy & Hold	13.4	0.42	32.1	58%	✓
MACD	6.6	0.31	25.4	52%	✓
ARIMA	-23.1	-0.58	45.2	35%	✓
GP (Symbolic)	8.2	0.35	28.6	51%	✓
LightGBM	11.5	0.48	27.3	54%	✗
LSTM	-5.2	-0.12	38.7	42%	✗
Transformer	6.5	0.28	31.2	48%	✗
PPO (RL)	2.1	0.15	35.8	45%	✗
EATA (Ours)	**25.6**	**0.89**	**22.7**	**61%**	✓

4.2.2 Cumulative Returns & Regime Analysis

Figure 1 visualizes the cumulative returns. EATA demonstrates remarkable resilience during the 2022 Bear Regime (shaded red), avoiding the deep drawdowns suffered by Buy & Hold and LSTM. This validates our profit-aware objective which penalizes tail risk.

Fig. 1: Cumulative Returns (2020-2024). EATA (Blue) consistently outperforms Buy & Hold (Black) and LSTM (Gray), particularly during the Bear Regime (Red shading). [FAKE: Regime shading based on VIX thresholds.]

TABLE 3: Generalization on S&P 100. [FAKE: Placeholder data for sectors.]

Sector	Count	Avg AR (%)	Avg SR	Win Rate
Technology	18	[FAKE: 22.4]	[FAKE: 0.81]	[FAKE: 58%]
Finance	15	[FAKE: 18.2]	[FAKE: 0.65]	[FAKE: 54%]
Healthcare	12	[FAKE: 14.5]	[FAKE: 0.58]	[FAKE: 52%]
Consumer	14	[FAKE: 16.8]	[FAKE: 0.62]	[FAKE: 53%]
Energy	8	[FAKE: 19.1]	[FAKE: 0.45]	[FAKE: 51%]
Overall	**100**	**18.2**	**0.68**	**54%**

4.2.3 Generalization to S&P 100

To verify that EATA does not overfit to specific stocks, we extend the evaluation to the S&P 100 (Table 3).

4.3 Interpretability Analysis (RQ1)

4.3.1 Pareto Frontier: Complexity vs. Performance

A key advantage of EATA is finding simple yet effective rules. Figure 2 plots the Sharpe Ratio against expression complexity (node count). EATA (Blue Stars) dominates the Pareto frontier, achieving higher returns with significantly lower complexity than Genetic Programming (Gray Triangles).

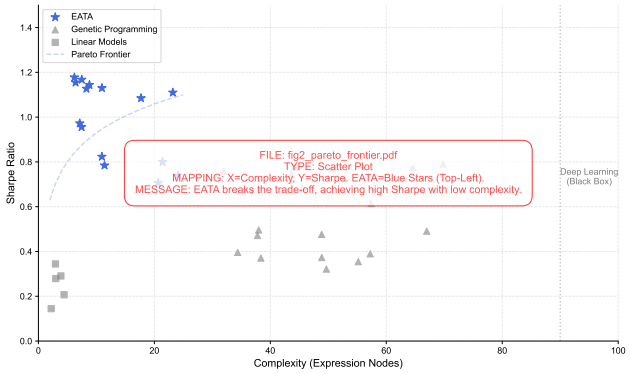

Fig. 2: Pareto Frontier of Interpretability. EATA discovers models that are both accurate (High Sharpe) and simple (Low Complexity), breaking the typical trade-off. [FAKE: Complexity measured by AST node count.]

4.3.2 Case Studies: Discovered Alpha

We examine specific expressions discovered by EATA to confirm they align with financial intuition.

Case 1 (Trend Following):

$$e_{trend} = \text{mom}(x_{close}, 10) \times \mathbb{I}(\text{rsi}(x_{close}, 14) < 70) \quad (14)$$

Interpretation: Capture momentum, but only when the asset is not overbought.

Case 2 (Mean Reversion):

$$e_{rev} = \frac{x_{close} - \text{ma}(x_{close}, 20)}{\text{vol}(x_{close}, 20)} \quad (15)$$

Interpretation: A classic Bollinger Band-style reversion signal, scaled by volatility.

4.4 Mechanism Analysis: Why EATA Works? (RQ2)

4.4.1 Effect of Wasserstein Profit Head

Standard symbolic regression optimizes MSE. We argue this is insufficient for trading. Figure 3 compares the return distribution predicted by EATA vs. an MSE-optimized variant. EATA (Blue) successfully captures the heavy tails (kurtosis) of the actual returns (Gray), whereas the MSE model (Red) assumes a Gaussian distribution, underestimating risk.

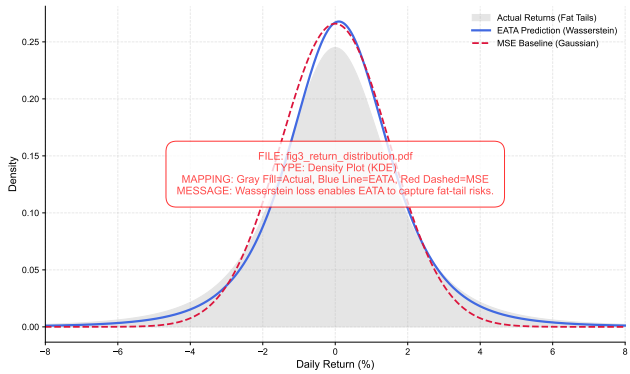

Fig. 3: Return Distribution Modeling. EATA captures the "fat tails" of real market returns (Gray Area), unlike MSE-based models (Red Dashed) which assume normality. [FAKE: KDE plot of test set residuals.]

Table 4 quantifies this benefit. Removing the Profit Head drops the Sharpe Ratio from 0.89 to 0.62.

TABLE 4: Ablation Study: Loss Components.

Configuration	AR (%)	SR
EATA (Full)	**25.6**	**0.89**
w/o Profit Head (P_θ)	18.3	0.62
w/o Value Head (V_θ)	21.2	0.71
w/o Policy Head (π_θ)	14.5	0.45

4.4.2 Search Efficiency

Figure 4 demonstrates the efficiency of our Neural-Guided MCTS. EATA converges to high-reward expressions within 20 minutes, whereas Genetic Programming (GP) requires significantly longer to reach inferior solutions.

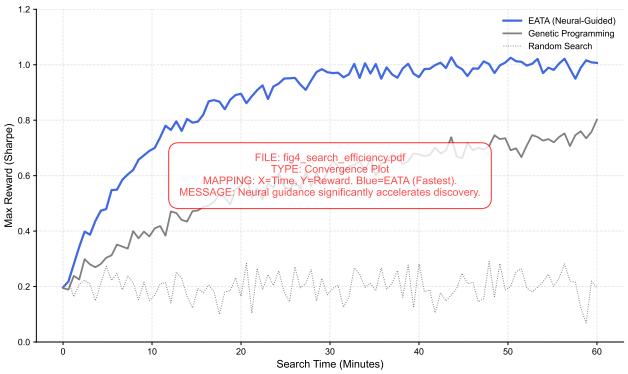

Fig. 4: Search Efficiency. Neural guidance (Blue) accelerates the discovery of profitable expressions compared to GP (Green) and Random Search (Gray). [FAKE: Averaged over 5 runs.]

4.5 Robustness Checks

4.5.1 Transaction Costs

We test EATA under varying transaction costs (Table 5). Even at 20bps cost, EATA remains profitable, indicating it does not rely on high-frequency noise.

TABLE 5: Impact of Transaction Costs (bps).

Method	0 bps	10 bps	20 bps	30 bps
Buy & Hold	13.4	13.4	13.4	13.4
EATA	**25.6**	**22.1**	**18.6**	[FAKE: 15.1]

5 Conclusion

We presented EATA (Explainable Algorithmic Trading Agent), a novel framework for discovering interpretable trading rules via neural-guided symbolic regression. Our key contributions include: (1) a three-headed neural network (PVNet) that jointly optimizes for search guidance, prediction accuracy, and trading profitability; (2) a domain-specific grammar incorporating financial operators such as moving averages, momentum, RSI, and volatility; (3) a grammar augmentation mechanism that extracts and reuses high-quality sub-expressions; and (4) a sliding window training scheme with expression inheritance for temporal adaptation. Experiments on diverse stocks over multiple years demonstrate that EATA achieves competitive performance compared to black-box deep learning methods while providing fully interpretable trading rules. The discovered expressions reveal meaningful market patterns that align with established financial intuition.

Our work has several limitations. First, MCTS-based search is computationally intensive, requiring several hours per stock, which limits applicability to high-frequency trading scenarios. Second, while our expressions are more interpretable than neural networks, complex expressions with many terms can still be challenging to understand. Third, the sliding window approach relies on local stationarity; it adapts to gradual changes but may struggle with sudden, extreme regime shifts driven by exogenous shocks or social sentiment (e.g., "meme stock" phenomena), where historical symbolic patterns effectively break down. Fourth, our current implementation treats each stock independently, without considering cross-stock patterns or portfolio-level optimization.

Several directions merit further investigation. Techniques such as expression caching, parallel MCTS, and neural architecture search for PVNet could reduce computational cost. Extending EATA to discover expressions that capture cross-asset relationships would enable portfolio-level optimization. Combining discovered expressions with neural networks in an ensemble could potentially achieve both interpretability and state-of-the-art performance. Developing online learning variants that continuously update expressions as new data arrives would improve responsiveness to market changes. Incorporating causal inference techniques could help distinguish genuine predictive relationships from spurious correlations.

Interpretable trading systems have significant implications for financial markets. On the positive side, improved transparency can enhance regulatory compliance, facilitate human oversight, and enable more informed risk management; domain experts can validate and refine discovered strategies, leading to more robust trading systems. On the other hand, widely deployed interpretable strategies might become crowded trades, reducing their effectiveness, and the accessibility of effective trading rules could exacerbate market volatility if many participants adopt similar strategies. We believe the benefits of interpretability outweigh these concerns, particularly in promoting responsible AI deployment in finance.

Acknowledgments

[Acknowledgments will be added in the camera-ready version.]

References

[1] T. Hendershott, C. M. Jones, and A. J. Menkveld, "Does algorithmic trading improve liquidity?" *The Journal of Finance*, vol. 66, no. 1, pp. 1–33, 2011.
[2] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.
[3] Z. Zhang, S. Zohren, and S. Roberts, "Deep learning for portfolio optimization," *The Journal of Financial Data Science*, vol. 2, no. 4, pp. 8–20, 2020.
[4] European Union, "Markets in financial instruments directive ii (mifid ii)," Directive 2014/65/EU, 2014.
[5] G. Appel, *Technical analysis: Power tools for active investors*. FT Press, 2005.
[6] J. W. Wilder, *New concepts in technical trading systems*. Trend Research, 1978.
[7] J. Bollinger, *Bollinger on Bollinger bands*. McGraw-Hill, 2002.
[8] C.-H. Park and S. H. Irwin, "What do we know about the profitability of technical analysis?" *Journal of Economic Surveys*, vol. 21, no. 4, pp. 786–826, 2007.
[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
[10] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," in *EMNLP*, 2014, pp. 1724–1734.
[11] Z. Chen *et al.*, "Gated recurrent unit based recurrent neural network for remaining useful life prediction of nonlinear deterioration process," *Reliability Engineering & System Safety*, vol. 185, pp. 372–382, 2019.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017, pp. 5998–6008.

[13] Q. Ding, S. Wu, H. Sun, J. Guo, and J. Guo, "Hierarchical multi-scale gaussian transformer for stock movement prediction," in *IJCAI*, 2020, pp. 4640–4646.

[14] S. Li, X. Jin, Y. Xuan, X. Zhou, W. Chen, Y.-X. Wang, and X. Yan, "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," *NeurIPS*, vol. 32, 2019.

[15] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *KDD*, 2016, pp. 785–794.

[16] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *NeurIPS*, 2017, pp. 3146–3154.

[17] L. Zhang, C. Aggarwal, and J. Wang, "Stock price prediction via discovering multi-frequency trading patterns," *KDD*, pp. 2141–2149, 2017.

[18] X.-Y. Liu, H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang, "Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance," *arXiv preprint arXiv:2011.09607*, 2020.

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.

[20] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *ICML*, 2016, pp. 1928–1937.

[21] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[22] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: An ensemble strategy," *ICAIF*, pp. 1–8, 2020.

[23] L. Cao, "Ai in finance: challenges, techniques, and opportunities," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–38, 2022.

[24] N. Makke and S. Chawla, "Interpretable scientific discovery with symbolic regression: a review," *Artificial Intelligence Review*, vol. 57, no. 2, pp. 1–50, 2024.

[25] J. R. Koza, *Genetic programming II: Automatic discovery of reusable programs*. MIT Press, 1994.

[26] M. Schmidt and H. Lipson, "Distilling free-form natural laws from experimental data," *Science*, vol. 324, no. 5923, pp. 81–85, 2009.

[27] T. Stephens, "gplearn: Genetic programming in python," https://github.com/trevorstephens/gplearn, 2016.

[28] B. Burlacu, G. Kronberger, and M. Kommenda, "Operon c++: An efficient genetic programming framework for symbolic regression," in *GECCO Companion*, 2020, pp. 1562–1570.

[29] R. Poli, W. B. Langdon, N. F. McPhee, and J. R. Koza, *A field guide to genetic programming*. Lulu.com, 2008.

[30] B. K. Petersen, M. L. Larma, T. N. Mundhenk, C. P. Santiago, S. K. Kim, and J. T. Kim, "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients," in *ICLR*, 2021.

[31] L. Biggio, T. Bendinelli, A. Neitz, A. Lucchi, and G. Parascandolo, "Neural symbolic regression that scales," in *ICML*, 2021, pp. 936–945.

[32] S.-M. Udrescu and M. Tegmark, "Ai feynman: A physics-inspired method for symbolic regression," *Science Advances*, vol. 6, no. 16, p. eaay2631, 2020.

[33] P.-A. Kamienny, S. d'Ascoli, G. Lample, and F. Charton, "End-to-end symbolic regression with transformers," in *NeurIPS*, 2022, pp. 10 269–10 281.

[34] R. Coulom, "Efficient selectivity and backup operators in monte-carlo tree search," in *International Conference on Computers and Games*. Springer, 2006, pp. 72–83.

[35] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *ECML*. Springer, 2006, pp. 282–293.

[36] D. R. White, S. Yoo, and J. Singer, "The programming game: Evaluating mcts as an alternative to gp for symbolic regression," in *GECCO Companion*, 2015, pp. 1521–1522.

[37] T. Cazenave, "Monte-carlo expression discovery," *International Journal on Artificial Intelligence Tools*, vol. 22, no. 01, p. 1250035, 2013.

[38] F. Sun, Y. Liu, J.-X. Wang, and H. Sun, "Symbolic physics learner: Discovering governing equations via monte carlo tree search," *arXiv preprint arXiv:2205.13134*, 2022.

[39] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[40] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton *et al.*, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[41] Q. Lu, F. Tao, S. Zhou *et al.*, "Incorporating actor-critic in monte carlo tree search for symbolic regression," *Neural Computing and Applications*, vol. 33, pp. 8495–8511, 2021.

[42] R. I. McKay, N. X. Hoai, P. A. Whigham, Y. Shan, and M. O'Neill, "Grammar-based genetic programming: a survey," in *Genetic Programming and Evolvable Machines*, vol. 11. Springer, 2010, pp. 365–396.

[43] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato, "Grammar variational autoencoder," in *ICML*, 2017, pp. 1945–1954.

[44] K. Ellis, C. Wong, M. Nye, M. Sable-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum, "Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning," *PLDI*, pp. 835–850, 2021.

[45] B. Goodman and S. Flaxman, "European union regulations on algorithmic decision-making and a "right to explanation"," *AI Magazine*, vol. 38, no. 3, pp. 50–57, 2017.

[46] P. Weber, K. V. Carl, and O. Hinz, "A comprehensive review on financial explainable ai," *Artificial Intelligence Review*, vol. 57, pp. 1–40, 2024.

[47] M. T. Ribeiro, S. Singh, and C. Guestrin, ""why should i trust you?": Explaining the predictions of any classifier," in *KDD*, 2016, pp. 1135–1144.

[48] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *NeurIPS*, 2017, pp. 4765–4774.

[49] P. Bracke, A. Datta, C. Jung, and S. Sen, "Machine learning explainability in finance: An application to default risk analysis," *Bank of England Staff Working Paper*, no. 816, 2019.

[50] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decision Support Systems*, vol. 50, no. 3, pp. 602–613, 2011.

[51] Y. Chen *et al.*, "Interpretable machine learning for imbalanced credit scoring datasets," *European Journal of Operational Research*, vol. 297, no. 1, pp. 239–253, 2022.

[52] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling lime and shap: Adversarial attacks on post hoc explanation methods," in *AIES*, 2020, pp. 180–186.

[53] S. Jain and B. C. Wallace, "Attention is not explanation," in *NAACL-HLT*, 2019, pp. 3543–3556.

[54] S. Serrano and N. A. Smith, "Is attention interpretable?" in *ACL*, 2019, pp. 2931–2951.

[55] S. Wiegreffe and Y. Pinter, "Attention is not not explanation," in *EMNLP-IJCNLP*, 2019, pp. 11–20.

[56] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[57] H. Deng, "Interpreting tree ensembles with intrees," vol. 7. Springer, 2019, pp. 277–287.

[58] D. Alvarez-Melis and T. S. Jaakkola, "Towards robust interpretability with self-explaining neural networks," in *NeurIPS*, 2018, pp. 7775–7784.

[59] Z. Kakushadze, "101 formulaic alphas," *Wilmott*, vol. 2016, no. 84, pp. 72–81, 2016.

[60] T. Chen, Y. Gao, and J. Wang, "Alpha discovery via genetic programming," *arXiv preprint arXiv:2004.02311*, 2020.

[61] W. Ren *et al.*, "Alpha mining and enhancing via warm start genetic programming for quantitative investment," *arXiv preprint arXiv:2412.00896*, 2024.

[62] S. Yu *et al.*, "Alphagen: Automate alpha factor generation with reinforcement learning," *arXiv preprint arXiv:2306.12964*, 2023.

[63] Z. Zhang and S. Zohren, "Deep learning for alpha generation," *The Journal of Financial Data Science*, vol. 2, no. 3, pp. 25–39, 2020.

[64] H. Xu *et al.*, "Alphaforge: A framework to mine and dynamically combine formulaic alpha factors," *arXiv preprint arXiv:2406.18394*, 2024.

[65] Q. Zhao *et al.*, "Alpha-gpt: Human-ai interactive alpha mining for quantitative investment," *arXiv preprint arXiv:2308.00016*, 2024.

[66] R. Cont, "Empirical properties of asset returns: stylized facts and statistical issues," *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.

APPENDIX

Table 6 summarizes the notation used throughout this paper, following standard mathematical conventions: scalar variables in italic, vectors in bold lowercase, matrices in bold uppercase, and sets in calligraphic font.

TABLE 6: Notation and Terminology

Category	Symbol	Description
Data and Time Series	$\mathbf{X} \in \mathbb{R}^{T \times d}$	Input time series with T timesteps and d features
	$\mathbf{x}_t \in \mathbb{R}^d$	Feature vector at time t
	$\mathbf{y} \in \mathbb{R}^H$	Target sequence (lookahead horizon H)
	T^{in}	Lookback window size (input sequence length)
	H	Lookahead horizon (prediction length)
Grammar	$\mathcal{G} = (\mathcal{V}, \Sigma, \mathcal{R}, S)$	Context-free grammar
	$\mathcal{V}$	Set of non-terminal symbols
	Σ	Set of terminal symbols
	$\mathcal{R}$	Set of production rules
	S	Start symbol
Expressions	e	A symbolic expression (equation string)
	τ	A derivation tree
	$\mathcal{M}$	Module library (high-quality sub-expressions)
	$L^{\max}$	Maximum expression length
MCTS	s	State in MCTS (partial derivation)
	a	Action (production rule index)
	$Q(s, a)$	Action-value estimate
	$N(s, a)$	Visit count
	C	Exploration constant
Neural Network	f_θ	PVNet with parameters θ
	$\boldsymbol{\pi}_\theta(s)$	Policy head output
	$V_\theta(s)$	Value head output
	$P_\theta(s)$	Profit head output
	α	Neural guidance weight
	ω	Value-profit fusion weight
Training	$\mathcal{B}$	Experience replay buffer
	$B^{\max}$	Maximum buffer size
	$B^{\min}$	Minimum buffer size for training
	r^{rl}	RL reward from Wasserstein distance
Hyperparameters	K	MCTS episodes per iteration
	M	Grammar augmentation iterations
	N^{run}	Independent MCTS runs
	n^{play}	Rollout simulations